

Home Assignment 4 in SSY345 - Analysis Part

Basic information

This home assignment is related to the material in lecture 7 and 8. In the analysis part we want you to use the toolbox that you have developed and apply it to a practical scenario. Associated with each scenario is a set of tasks that we would like you to perform. It is important that you comment your code such that it is easy to follow by us and your fellow students. Poorly commented code will result in deduction of POE. Your code should be exported as a `.txt` and uploaded as part of your submission before the deadline. The purpose is to make feedback from your peers possible and to enable plagiarism analysis (Urkund) of all your submissions. The result of the tasks should in general be visualised and compiled together in a report (`.pdf`-file). A template for the report can also be found on course homepage. Note that, it is sufficient to write short concise answers to the questions but they should be clearly motivate by what can be seen in the figures. Only properly referenced or captioned figures such that it is understandable what will result in POE. Also, all the technical terms and central concepts to explain an answer should be used without altering their actual meaning. The report should be uploaded on the course homepage before the deadline.

1 Smoothing

In this task, we'll familiarize with the RTS smoother.

We build on task 3 of HA3 by performing RTS smoothing on a measurement sequence, and then compare with the filter outputs from the same data.

Generate a true state sequence

$$\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{800} \quad (1)$$

using the code in Table 1. If you plot the positions, you will see that the sequence consists of a straight line, followed by two 90-degree turns followed by another straight line. We proceed to use the dual bearing measurement model and the true sequence to generate measurement sequences

$$\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_{800}. \quad (2)$$

The scenario is identical to task 3 of HA3, the various parameters are listed here for completeness:

Parameter	Notation	Value
Sampling interval	T	0.1 s
Prior mean	$\mathbf{x}_0$	$[0, 0, 0, -\pi/2, 0]^T$
Prior covariance	$\mathbf{P}_0$	$\text{diag}([10^2, 10^2, 10^2, (\frac{5\pi}{180})^2, (\frac{5\pi}{180})^2])$
Sensor 1 pos.	$\mathbf{s}_1$	$[160, -300]^T$
Sensor 2 pos.	$\mathbf{s}_2$	$[420, -300]^T$
Sensor 1 noise	$\sigma_\varphi^{(1)}$	$0.5\pi/180$
Sensor 2 noise	$\sigma_\varphi^{(2)}$	$0.5\pi/180$

The process noise covariance $\mathbf{Q}$ should be set to the well-tuned process covariance matrix resulting from your work in task 3 of HA3. You may of course change the value if your tuning in HA3 did not give a satisfactory result.

Task: Filter the measurement sequence using your favorite nonlinear Kalman filter using the coordinated turn model, and run the corresponding smoother on the measurement sequence and filtering densities.

- a Plot the sensor positions, the positions corresponding to the measurements, the true position sequence the filtered and smoothed position trajectories as well as their 3σ -covariance contours for every 10th filter and smoothing estimate. (1.0 p)

- Compare the shape of the filter- and smoother trajectories and comment on your observations.
 - What are the differences between filter and smoothing error covariances in the same time instance? Explain why there's a difference.
- b Modify a measurement at one time instance $k = 400$ to manually introduce an outlier. Run your smoother on the modified measurement sequence and illustrate the resulting trajectories along with the measurement sequence in Cartesian coordinates in an area surrounding the outlier measurement. How well does the filter and smoother cope with the introduced outlier? Explain observed differences. (0.5 p)
- Hint: to introduce an outlier, one approach could be to add some additional error to the existing measurement $\mathbf{y}'_{400} = \mathbf{y}_{400} + \Delta/\sqrt{2}[1, -1]^T$ where Δ is the error magnitude in unit radians.*

Table 1: Generate true track

```
% Sampling period
T = 0.1;
% Length of time sequence
K = 800;
% Allocate memory
omega = zeros(1,K+1);
% Set turn-rate at turns
omega(200:350) = pi/301/T;
omega(450:600) = pi/301/T;
% Initial state
x0 = [0 0 20 -pi/2 0]';
% Allocate memory
X = zeros(length(x0),K+1);
X(:,1) = x0;
% Create true track
for i=2:K+1
    % Simulate
    X(:,i) = coordinatedTurnMotion(X(:,i-1), T);
    % Set turn-rate
    X(5,i) = omega(i);
end
```

2 Particle filters for linear and Gaussian systems

In this task, you will study the basic properties of particles in a linear and Gaussian setting.

Consider the following linear and Gaussian state space model:

$$x_k = x_{k-1} + q_{k-1} \quad (3)$$

$$y_k = x_k + r_k, \quad (4)$$

where the motion and measurement noises, q_{k-1} and r_k , are zero mean Gaussian random variables with variances $Q = 1.5$ and $R = 0.5$, respectively, and the initial prior is $p(x_0) = \mathcal{N}(x_0; 2, 8)$.

- a) Generate a trajectory and measurement sequence and run a KF, a PF without resampling and a PF with resampling to recursively compute $p(x_k|y_{1:k})$ and its first two moments. Note that for the bootstrap particle filter with resampling, the resampling is usually performed each time step. (1.0 p)

- Compare the performance of the two PFs with the KF in terms of root mean square error (RMSE). How many particles do you need in order for your two PFs to perform approximately as well as the KF? Use the value you find most suitable in the next subproblems. Comment on your results and observations.

Hint: in general $\text{RMSE} = \sqrt{\frac{1}{N} \sum_{n=1}^N \text{error}_n^2}$.

- Plot the true trajectory and the measurement sequence as well as $\mathbb{E}[x_k|y_{1:k}]$ and $\text{Cov}[x_k|y_{1:k}]$ for the three filters over time. *Hint: one way to visualize $\mathbb{E}[x_k|y_{1:k}]$ and $\text{Cov}[x_k|y_{1:k}]$ is to use `errorbar(x, y, err)` in MATLAB.*
- Illustrate the approximations of the posterior densities produced by the PFs together with the one output by the KF at three different time instances; one close to the beginning, one in the middle and one close to the end. Comment on your results and observations. *Hint: you may visualize the density produced by the PF without resampling using the `stem()`-function in MATLAB. For the PF with resampling, it's recommended to use `histogram(..., 'Normalization', 'pdf')`.*

- b) Now we'll study what happens if we use an incorrect prior at time $k = 0$. Keep the same process- and measurement model for generating the true state sequence and measurements, but now filter the measurements using

the prior $p(x_0) = \mathcal{N}(x_0; -20, 2)$. Observe how fast the estimates produced by the three different filters can “catch up” with the true sequence. Comment on your findings. (0.25 p)

- c) Now repeat subtask b) using particle filters with and without resampling with $N = 10$ particles (we choose a small number of particles here simply for visualization purposes). Illustrate the individual particle trajectories using the function defined in `plotPartTrajs.m` by passing it as a function handle to your particle-filtering routine. Compare the two filters and comment on your results. (0.25 p)

3 Bicycle tracking in a village

In this task, we'll use the particle filter to solve a more practical problem.

Suppose we are interested in tracking a bicycle in a village and that we have an accurate map of the buildings in the village, see Fig. 1. We know that the bicycle is always on the road between the buildings. The tracking is based on (two-dimensional) measurements of the difference in position. The following problems should be considered:



Figure 1: This figure illustrates a map of a village in a cartesian coordinate system.

- a) Generate a sequence of positions, $\mathbf{x}_k^p$, using *MapProblemGetPoint.m*. Run the file and use the left mouse button to generate a trajectory of the bicycle, press Return to stop collecting measurements. Try to make it resemble the movement of a real bicycle. (0.15 p)
- b) Generate velocity measurements

$$\mathbf{y}_k = \mathbf{x}_k^v + \mathbf{r}_k, \quad (5)$$

where $\mathbf{x}_k^v = \mathbf{x}_k^p - \mathbf{x}_{k-1}^p$ and $\mathbf{r}_k \sim \mathcal{N}(\mathbf{0}, \sigma_r^2 \mathbf{I}_{2 \times 2})$ (0.15 p).

- c) You can assume that the bicycle does not enter any buildings. Discuss how the information from the map can be used in a filter. Is it easier to

view the map as a measurement or as a component in the motion model?
Try to motivate. (0.2 p)

- d) Construct a PF algorithm with the ability to track both the position and the velocity of the bicycle. Assume that the initial position of the bicycle is known. You need to tune both the measurement and motion noise in order to obtain good performance. It is advisable to illustrate your particles in the village in each recursion in order to visualize the filter performance. (1.5 p)
- e) Generalize the PF such that it can track the bicycle without any knowledge about its initial position. Is your filter able to position the bicycle? If so, which information is it that enables it to do so? Please try to illustrate (or at least point out in words) the events when the filter manages to rule out large chunks of particles. (1.0 p)

Hint: Ignoring the information from the map, it may be reasonable to use a 2-dimensional constant velocity equation as motion model and (5) as measurement model (four state variables). How well that model fits the data depends on the trajectories that you create (to some extent it is up to you!)

Hint: You have access to a Matlab-function isOnRoad.m that checks which points are on a road in the village.